Lecture - Retrieval Agents and Lakebase Search on Databricks

Duration: 30 minutes

Overview

This lecture introduces retrieval agents and AI engineering on Databricks. We begin with why prompting alone isn't enough and how Retrieval Augmented Generation (RAG) fills the gap. We then trace the evolution from prompt engineering to context engineering to harness engineering, and survey Databricks' retrieval approaches — from managed Genie Agents (recommended) and Knowledge Assistants (legacy), to fully custom code-first agents. Finally, we introduce Lakebase, Databricks' managed Postgres database, and Lakebase Search, which brings hybrid vector and keyword search directly into your operational database for co-located retrieval.

Learning Objectives

By the end of this lecture, you will be able to:

1. Explain what retrieval agents are and why they are needed
2. Distinguish prompt engineering, context engineering, and harness engineering
3. Describe Genie Agents as the simplest managed retrieval solution on Databricks
4. Identify the knowledge source types available to a Knowledge Assistant
5. Compare declarative vs. code-first approaches to building retrieval agents
6. Explain what Lakebase is and its key capabilities for AI applications
7. Describe Lakebase Search and when to use it vs. AI Search

A. Introduction to Retrieval Agents

A1. Why Retrieval Agents?

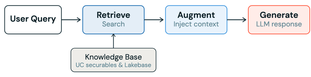
Large Language Models (LLMs) are powerful, but they have hard limits. No amount of prompt refinement can overcome:

Knowledge Cutoffs	Hallucination	Missing Private Context
The model doesn't know about events after its training date.	The model fabricates plausible-sounding facts when it lacks real data.	No access to your organization's proprietary documents.

Retrieval Augmented Generation (RAG) addresses these limits by injecting relevant external data into the model's context at query time.

Instead of relying on frozen training data, a RAG system **retrieves** relevant documents, **augments** the prompt with them, and lets the model **generate** a grounded response.

A **retrieval agent** is a concrete implementation of this pattern that handles query routing, retrieval orchestration, and context assembly within a real system.

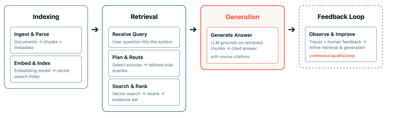


> **Learn More**
> See the Databricks documentation on Retrieval Augmented Generation (AWS | Azure | GCP) for a deeper dive.

A2. The Common Retrieval Pipeline

Regardless of the framework or product you choose, every RAG system follows the same four-stage workflow.

Understanding these stages helps you evaluate managed solutions and know what you're building when you go code-first.



▼ What are the four stages of a retrieval pipeline?

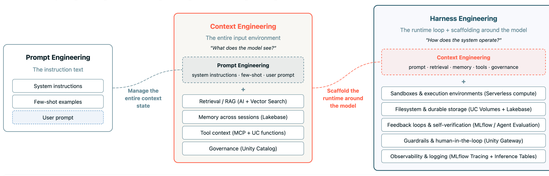
Every RAG system — whether managed or custom — flows through four stages: two offline (preparing the index) and two online (handling the request), with a feedback loop that improves quality over time.

1. **Indexing**: Ingest source documents, **parse, chunk, embed, and index** them into a searchable store (plus normalize metadata).
2. **Retrieval**: Receive the user query, **plan retrieval** by selecting sources and crafting sub-queries, then **search and rank** results from one or more indexes.
3. **Generation**: Use an LLM to **generate a grounded answer** from the retrieved context, including **source citations** back to the original documents.
4. **Feedback loop**: Use tracing, evaluation (LLM judges, human review), and user feedback to continuously refine both retrieval behavior and answer quality.

In Section C, you'll see how **Genie Agents** and **Knowledge Assistants** handle these stages for you, and how **custom agents** let you own each stage directly.

B. Introduction to AI Engineering

B1. From Prompt Engineering to Context Engineering to Harness Engineering



▼ What's the difference between prompt, context, and harness engineering?

Prompt engineering focuses on crafting the instruction text sent to a model. It involves choosing the right words, formatting, and examples. It's tactical and operates at the level of a single query.

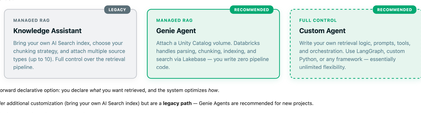
Context engineering is broader. It's the art and science of designing the entire *information environment* a model receives at inference time: not just the question, but all the supporting data, instructions, history, and constraints that shape the model's decision. It answers one question: **"What does the model see?"**

Harness engineering is broader still. It's the entire *scaffolding around the model* — the runtime loop that executes each conversation turn, the source of truth codified in-repo, the deterministic guardrails, and the workflow automation that make an agent reliable in production. It answers a different question: **"How does the system around the model operate?"** Context engineering is one component of it — the Foundry framing calls this **"Context Engineering++"**: context engineering *plus* workflow engineering, spec-driven development, and deterministic enforcement of invariants.

Think of it this way: prompt engineering is writing a good question on an exam. Context engineering is designing the entire exam room — the reference materials on the desk, the instructions on the board, the rules about what's allowed. Harness engineering is running the whole institution around it — writing the rules, invigilating, grading, and re-running the exam reliably every term. Context engineering makes the agent *smart*; harness engineering makes the agent *reliable and repeatable*.

C. Retrieval Approaches on Databricks

Databricks offers a spectrum of retrieval approaches — from fully managed to fully custom. **Genie Agents are the recommended managed path for RAG**; Knowledge Assistants remain available but are a legacy option. The code-first path gives you complete control.



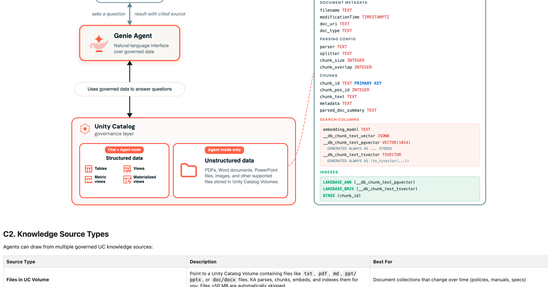
Genie Agents are the go-forward declarative option: you declare what you want retrieved, and the system optimizes *how*.

Knowledge Assistants offer additional customization (bring your own AI Search index) but are a **legacy path** — Genie Agents are recommended for new projects.

Custom agents are code-first: you own the retrieval logic end to end.

C1. Genie Agents: Attach a Volume, Get Search

A **Genie Agent** is the fastest managed path for retrieval on Databricks. In the simplest case, you attach a Unity Catalog volume containing your documents and Databricks handles the rest. But the more complete picture is that a Genie Agent sits on top of **Unity Catalog governed data**: structured data works in both Chat and Agent mode, while unstructured volume files are available in Agent mode.



C2. Knowledge Source Types

Agents can draw from multiple governed UC knowledge sources:

Source Type	Description	Best For
Files in UC Volume	Point to a Unity Catalog Volume containing files like `txt`, `pdf`, `md`, `ppt/pptx`, or `doc/docx`. Files. KA parses, chunks, embeds, and indexes them for you. Files >50 MB are automatically skipped.	Document collections that change over time (policies, manuals, specs)
AI Search Index	Attach a pre-built AI Search index (using embedding models like `databricks-gte-large-en`). You own the parsing/chunking/indexing pipeline.	Custom RAG pipelines where you already manage the index and schema
Files in UC Table / File Table	Use a UC table (e.g. **streaming table** or have **Change Data Feed enabled**) with a `content` column (BINARY/STRING) storing file contents plus a `_metadata` / `metadata` struct with the path/name/size/mod time.	Documents ingested via connectors (SharePoint, Google Drive, Jira, Confluence) that land as file tables

C3. Declarative vs. Code-First

Databricks offers two approaches to building retrieval agents:

	Declarative (Genie Agent)	Code-First (Custom Agents)
Approach	Declare what you want, the system learns and optimizes how	Write retrieval logic, prompts, tools, and orchestration yourself
Setup time	Minutes	Hours to days
Customizability	Configuration- and feedback-driven	Essentially unlimited
Optimization	Automatic via feedback loops and platform research upgrades	Manual tuning and experimentation
Best for	High-quality Q&A over docs, fast prototyping	Novel architectures, complex multi-step / tool-heavy workflows

This course focuses on the **declarative** path. We'll explore the components that understand the retrieval building blocks (document parsing, chunking, and AI Search), then bring them together using Genie Agents as the recommended managed approach.

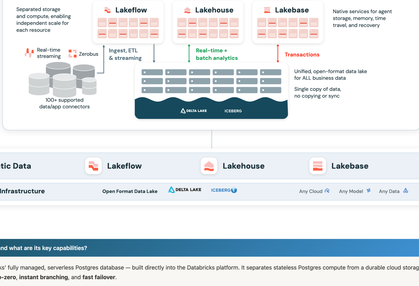
Genie IS CODE

Want to know more about Databricks features? Ask Genie Code. Click on the genie icon and begin querying.
For example, click the **Copy** button below and paste into **Genie Code**.

```
How can Genie Agents be used with Databricks Apps?
```

Copy

D. Introduction to Lakebase



▼ What is Lakebase and what are its key capabilities?

Lakebase is Databricks' fully managed, serverless Postgres database — built directly into the Databricks platform. It separates stateless Postgres compute from a durable cloud storage layer, enabling **autoscaling**, **scale-to-zero**, **instant branching**, and **fast failover**.

- **Transactional workloads** — Build low-latency OLTP applications alongside your lakehouse data, connected via Databricks Apps or any standard Postgres driver.
- **Lakehouse sync** — Sync Unity Catalog Delta tables into Lakebase so applications can query them at sub-millisecond latency.
- **AI & agent state** — Use Lakebase as an online feature store for ML models or as a state and memory store for AI agents.
- **Native Unity Catalog integration** — Governance, permissions, and identity flow seamlessly across the platform.

With that foundation in place, the next question is: how do we search this data efficiently? That's where **Lakebase Search** comes in.

E. Lakebase Search for Retrieval

E. Lakebase Search for Retrieval

Managed retrieval products like Genie Agents handle document-based RAG end to end. But when your data already lives in an operational database — customer reviews, product records, support tickets — you may want search **inside** that database so a single query can retrieve, JOIN, and enforce permissions in one transaction.

Lakebase Search (GA) adds hybrid vector and keyword search directly to Lakebase Postgres projects through two managed extensions:

EXTENSION

lakebase_vector

Approximate nearest-neighbor (ANN) search via the `lakebase_ann` index type. pgvector-compatible syntax. Uses IVF + RaBitQ quantization — supports 1B+ vectors per index, builds 50–100× faster than HNSW, and indexes survive scale-to-zero.

+

EXTENSION

lakebase_text

BM25 full-text keyword search via the `lakebase_bm25` index type. Compatible with PostgreSQL's `tsvector` types and operators. Top-K pushdown (Block-Max WAND) retrieves only the K most relevant results instead of scoring every match.

=

COMBINED

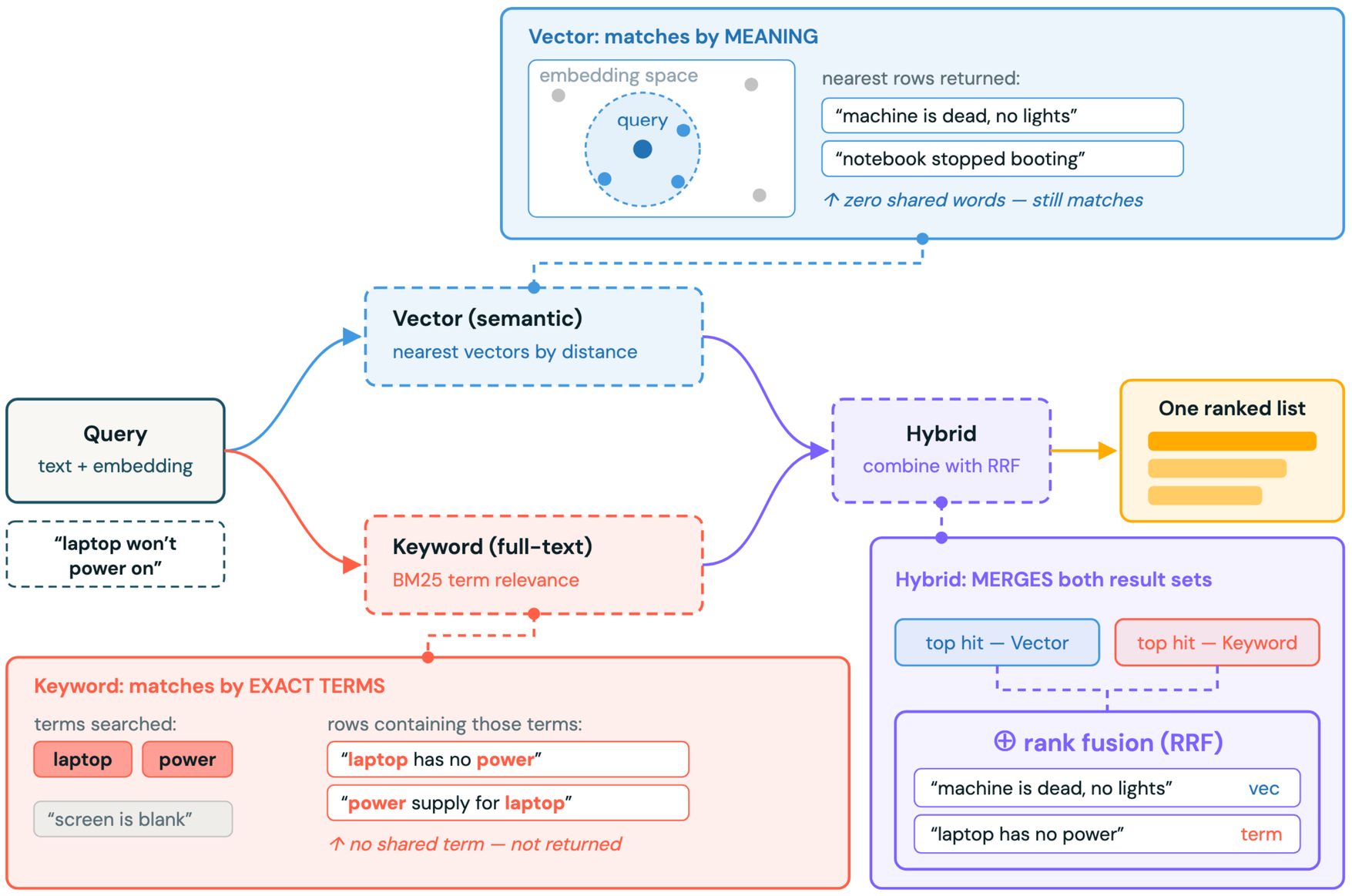
Hybrid Search

Run both searches and fuse results with reciprocal rank fusion (RRF). Reviews ranked well by *either* signal surface, and those ranked well by *both* rise to the top — the best of exact-term and semantic retrieval.

Learn More

See the Databricks documentation on Lakebase Search (AWS | Azure) for setup instructions and query examples.

E1. Lakebase Search Modes: Vector, Keyword, and Hybrid



Vector (semantic) search

Powered by `lakebase_vector`

- Finds rows whose **meaning** is closest — even with no shared words.
- Query with an embedding; index returns nearest vectors by distance.
- Best for natural-language questions, recommendations, and RAG.

Keyword (full-text) search

Powered by `lakebase_text`

- Ranks rows by how well they match your **exact terms**.
- Uses BM25 relevance scoring.
- Best for names, codes, and exact-term lookups where wording matters.

Hybrid search

Runs both, merges the results

- Combines both result sets into **one ranked list**.
- Returns semantically similar *and* exact-term matches together.
- Best when queries mix intent with specific terms.

Click a card to spotlight that path in the diagram — click it again to show everything.

© 2026 Databricks, Inc. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Privacy Policy | Terms of Use | Support